

Harness Engineering으로 완성하는 AI Agent 서비스 고도화

프롬프트를 넘어, 검증 가능한 AI Agent 팀을 설계하는
방법

좋은 답변 → 반복 가능한 품질



지금 우리는 어디를 지나고 있나?

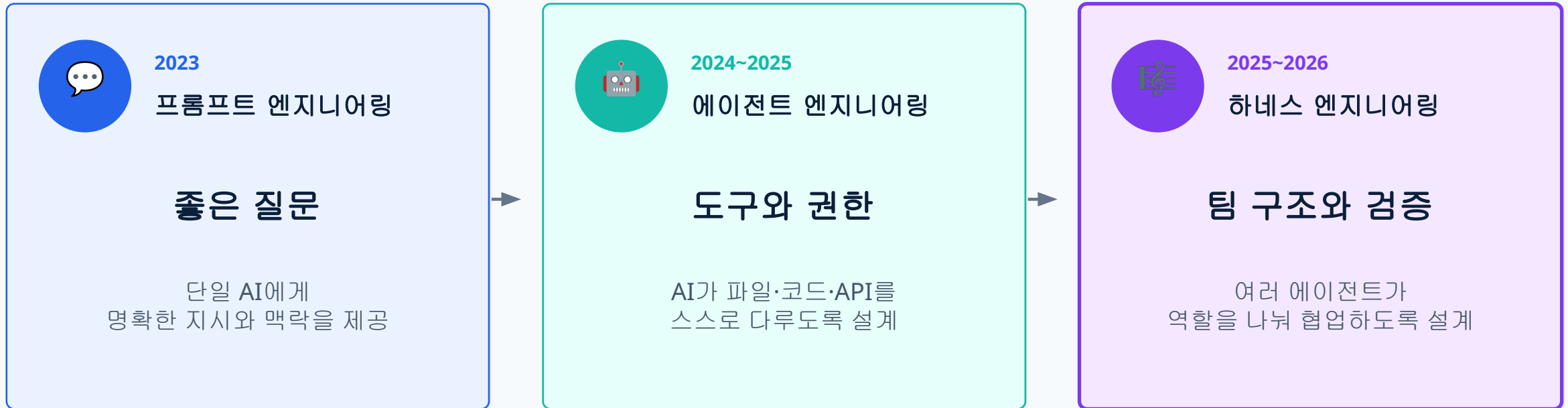
README 실습 흐름 안에서 하네스 설명이 들어가는 위치



핵심: 지금 만드는 것은 “서비스”만이 아니라, 서비스 품질을 검증하는 팀 구조입니다.

프롬프트 엔지니어링에서 하네스 엔지니어링으로

AI 활용은 좋은 질문을 넘어, AI 팀 구조와 검증 루프를 설계하는 방향으로 확장됩니다.



수업 메시지: 프롬프트는 개인기, 에이전트는 작업자, 하네스는 조직 설계입니다.

하네스란 무엇인가?

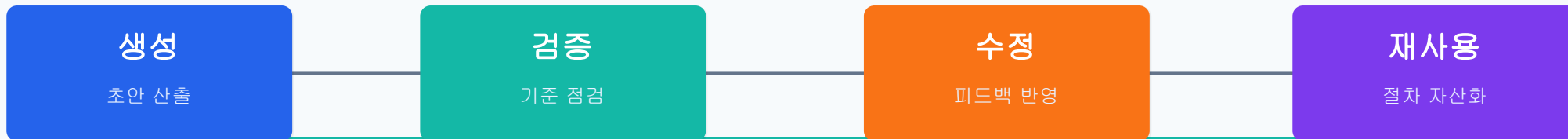
AI Agent가 일하는 환경을 설계하는 것: 역할, 도구, 권한, 검증 기준을 묶는 구조



Agent는 “무엇을/왜” 수행하는지에 집중하고, Harness는 “어떻게/어디서” 일할지 정합니다.

왜 지금 하네스가 필요한가?

좋은 답변보다 중요한 것은 반복 가능한 품질입니다.



금융 **AI Agent** 서비스에서는 “그럴듯함”이 아니라 검증 가능성이 핵심입니다.

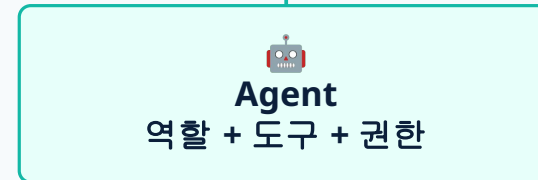
하네스는 오케스트라처럼 움직인다

오케스트레이터, 에이전트, 스킬이 함께 품질을 만듭니다.



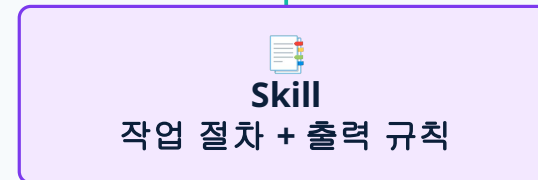
오케스트레이터

누가 언제 무엇을 할지
순서와 데이터 흐름을 조율



에이전트

특정 역할을 수행하는
전문가 단위



스킬

작업 매뉴얼처럼
입력·출력·제약을 명시

AI Agent 팀에도 진형이 있다

작업 성격에 따라 에이전트 팀의 구조를 다르게 설계합니다.

1

Pipeline

순차 작업

$A \rightarrow B \rightarrow C$

2

Fan-out/Fan-in

병렬 분석 후 통합

분산 $\rightarrow$ 통합

3

Expert Pool

전문가 선택

질문 성격별 라우팅

4

Producer-Reviewer

생성 후 검증

작성 $\leftrightarrow$ 리뷰

5

Supervisor

중앙 감독

동적 작업 분배

6

Hierarchical Delegation

계층적 위임

상위 $\rightarrow$ 하위

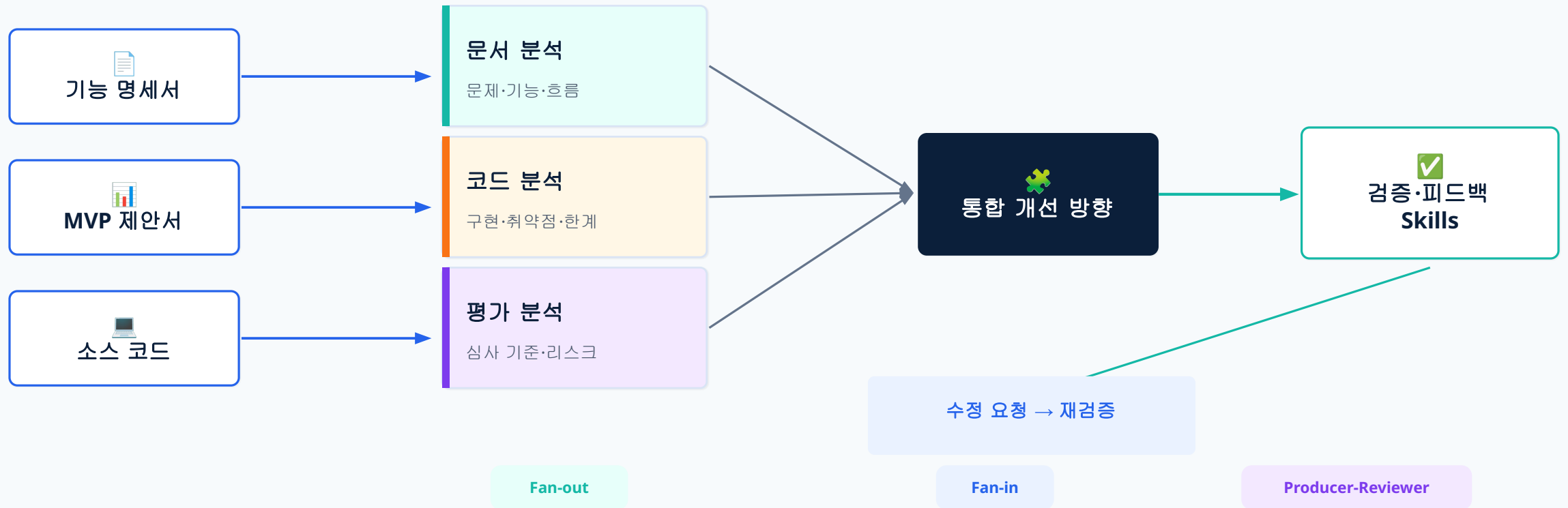
오늘 실습의 핵심 패턴

Fan-out/Fan-in + Producer-Reviewer

문서·코드·평가기준을 나눠 보고, 하나의 개선 방향으로 합친 뒤 다시 검증합니다.

오늘 실습은 어떤 하네스 패턴인가?

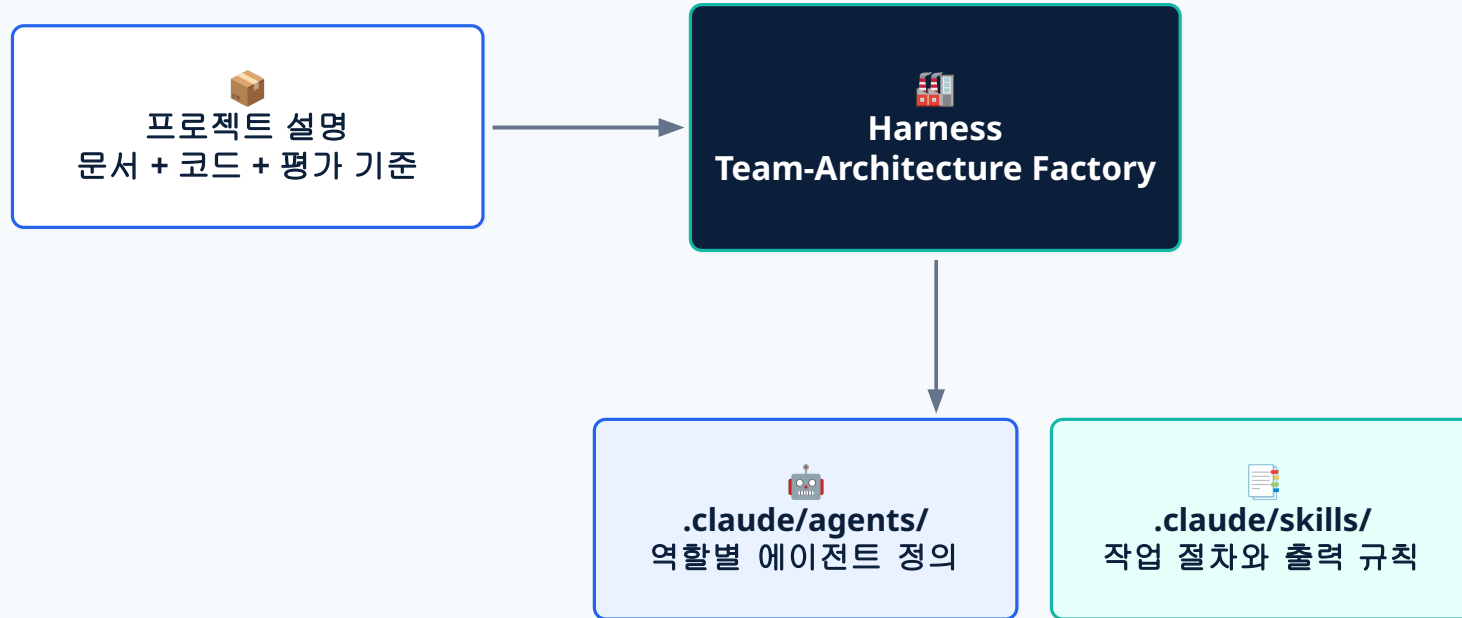
서비스 산출물을 만들고, 다시 AI Agent 팀이 검증하게 만드는 구조입니다.



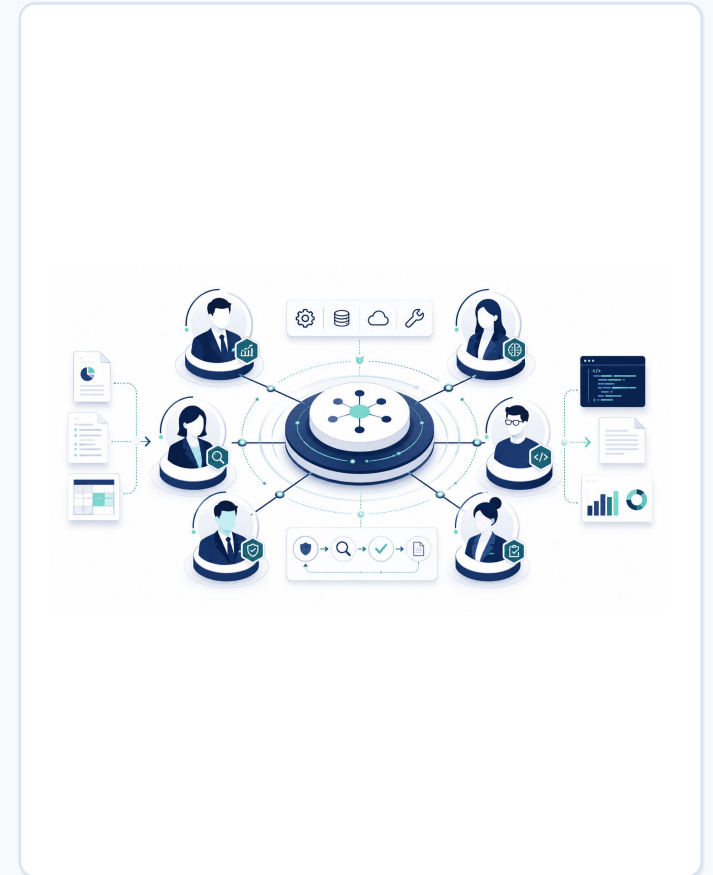
오늘의 `/harness`는 내 프로젝트 전용 검토팀을 구성하는 과정입니다.

Harness 플러그인이 해주는 일

프로젝트 설명을 바탕으로 에이전트 팀과 스킬을 생성하는 메타 스킬입니다.



수강생 관점에서는 한 줄 명령, 내부적으로는 "팀 설계 → 스킬 생성 → 검증 준비"입니다.



설치와 실행에서 꼭 봐야 할 체크포인트

명령을 입력하는 것보다, 실행 위치와 산출물 확인이 더 중요합니다.

Claude Code CLI

```
/plugin marketplace add revfactory/harness  
/plugin install harness@harness-marketplace  
/reload  
/harness
```

플러그인 설치 후 **/reload**를 잊지 않기



프로젝트 루트에서 실행

문서·코드가 있는 폴더에서 **/harness** 실행



입력 파일 확인

기능 명세서, MVP 제안서, 소스 코드가 함께 존재



요청 문장 구체화

“검증하고 고도화하는 하네스”처럼 목적 명시



결과 폴더 확인

.claude/agents/와 .claude/skills/ 생성 여부 확인

명령은 짧지만, 입력 컨텍스트가 부실하면 하네스도 부실해집니다.

하네스는 한 번에 마법처럼 생기지 않는다

Harness는 6단계 워크플로우로 팀 구조와 스킬을 만들어 갑니다.



수강생 화면에는 한 줄 명령처럼 보이지만, 실제 결과는 분석·설계·생성·검증의 누적입니다.

실행 후 어디를 봐야 하나?

생성된 폴더와 파일을 보고 “내 프로젝트 전용 하네스”인지 확인합니다.

your-project/

```
└── .claude/
    ├── agents/
    │   ├── analyst.md
    │   ├── builder.md
    │   └── qa.md
    └── skills/
        ├── analyze/
        │   └── SKILL.md
        ├── build/
        │   ├── SKILL.md
        │   └── references/
```

출력 폴더 구조

01

역할이 도메인에 맞는가?

준법, 보안, QA, 개선 제안 등 필요한 관점이 분리되었는지 확인

02

스킬에 절차가 있는가?

입력 규칙, 출력 형식, 검증 기준, 제약 조건이 명시되었는지 확인

03

평가 기준과 연결되는가?

문제정의, 금융 연계성, 구현 가능성, 리스크 관리 기준을 반영했는지 확인

04

다시 호출할 수 있는가?

생성된 스킬을 활용해 검증·고도화 요청을 반복할 수 있는지 확인

파일이 생긴 것만 보지 말고, “검증 기준이 실제 평가 기준과 이어지는지”를 봅니다.

금융 서비스에서는 “만들기”보다 “검증하기”가 중요하다

하네스는 아이디어를 검증 가능한 서비스 후보로 바꾸는 품질 게이트입니다.



업무 적합성

실제 금융 업무·고객 여정과 연결되는가?



고객 가치

효율, 리스크 감소, 고객 경험 개선이 보이는가?



기술 구현성

판단·행동·검증/개선 흐름이 있는가?



MVP 완성도

동작 가능한 최소 서비스로 확인 가능한가?



운영 리스크

개인정보, 보안, 환각, 책임소재 대응이 있는가?

이제 다시 실습으로 돌아가면 볼 것들

하네스 생성 결과를 눈으로 확인하고, 다음 개선 루프로 연결합니다.

확인 체크리스트

- ✓ .claude/agents/가 생성되었는가?
- ✓ .claude/skills/가 생성되었는가?
- ✓ 역할이 프로젝트 도메인에 맞게 나뉘었는가?
- ✓ 검증 기준이 기능 명세서·MVP 제안서·코드와 연결되는가?
- ✓ 생성된 스킬로 다시 서비스 개선 요청을 할 수 있는가?

다음에 다시 사용할 핵심 프롬프트

프로젝트 안에 기능 명세서,
MVP 제안서, 소스 코드를 읽고
AI Agent 서비스를 검증하고
고도화하는 하네스를 구성해줘.

하네스 검증 결과는 이후 Codex 발표자료 제작의 근거가 됩니다.

완성의 기준: 산출물이 “한 번 만든 파일”이 아니라, 반복해서 검증되는 작업 환경이 되는 것